

Prompting Tricks for LLM Reasoning: Comprehensive Evaluation Report

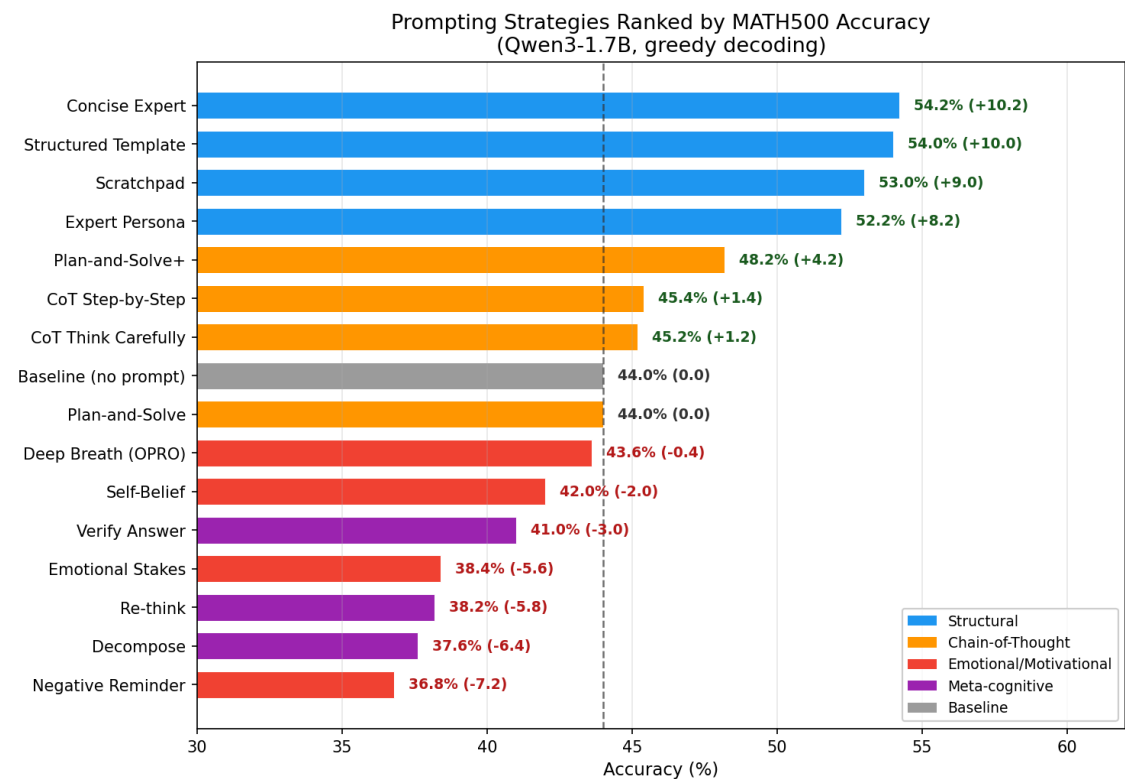
Run ID: 20260316_063839-prompting-tricks-reasoning **Date:** 2026-03-16 **Model:** Qwen3-1.7B (base, no fine-tuning) **Benchmark:** MATH500 (500 competition math problems, levels 1-5) **Inference:** SGLang, temperature=0.0, max_tokens=4096, greedy decoding

Executive Summary

We evaluated **16 prompting strategies** on MATH500 using Qwen3-1.7B to determine which prompt engineering techniques improve mathematical reasoning. Our key finding: **the best prompting tricks achieve +10.2% absolute improvement** over the bare baseline (54.2% vs 44.0%), but the picture is nuanced. Qwen3-1.7B already performs internal chain-of-thought reasoning via `<think>` tags regardless of the prompt, which diminishes the impact of explicit CoT instructions. Instead, **structural and task-framing prompts** (structured templates, scratchpads, expert personas) outperform traditional CoT by a wide margin. Motivational/emotional prompts consistently **hurt** performance.

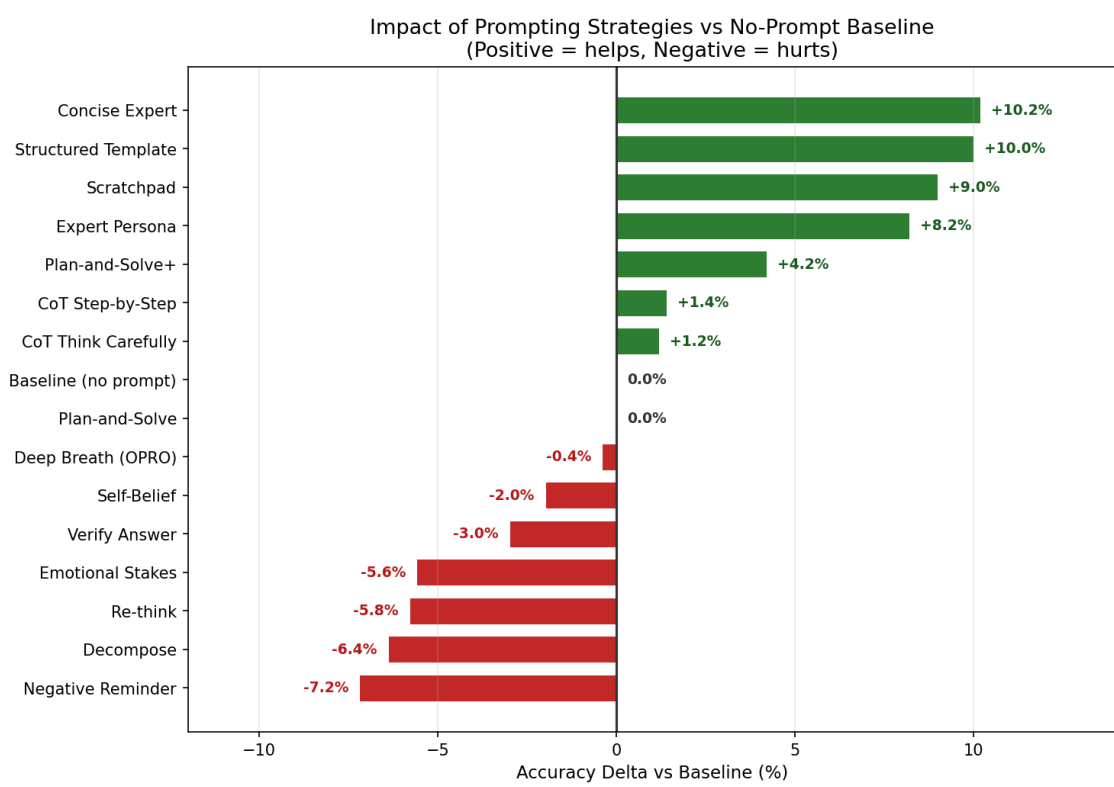
Results Summary

Figure 1: Overall Accuracy Ranking



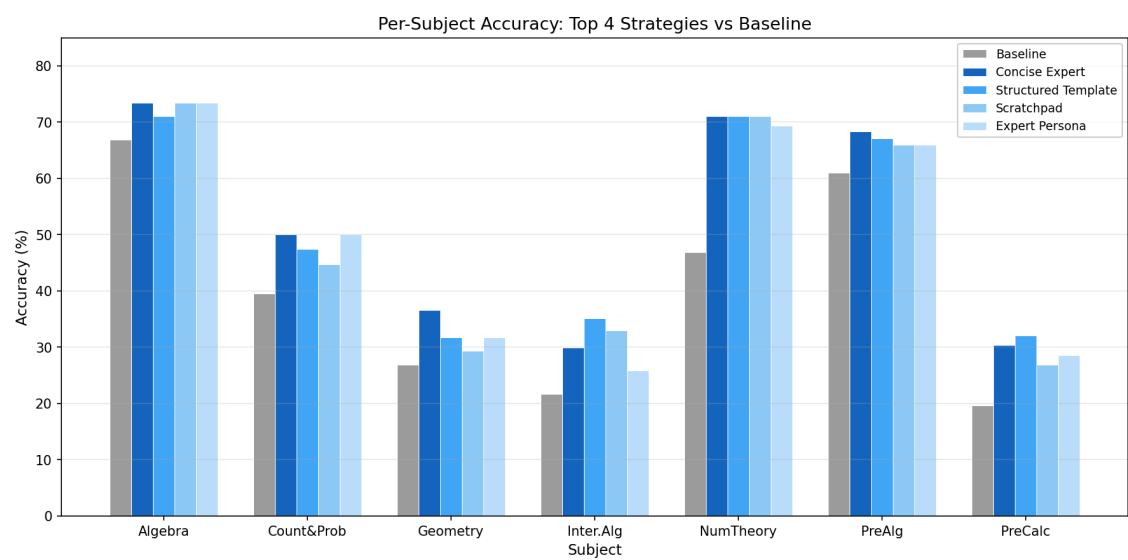
Rank	Strategy	Accuracy	Delta vs Baseline	Avg Response Length
1	concise_expert	54.2% (271/500)	+10.2%	7,538
2	structured_template	54.0% (270/500)	+10.0%	7,832
3	scratchpad	53.0% (265/500)	+9.0%	8,033
4	expert_persona	52.2% (261/500)	+8.2%	8,297
5	plan_and_solve_plus	48.2% (241/500)	+4.2%	8,250
6	cot_step_by_step	45.4% (227/500)	+1.4%	8,210
7	cot_think_carefully	45.2% (226/500)	+1.2%	8,298
8	baseline	44.0% (220/500)	—	8,272
9	plan_and_solve	44.0% (220/500)	+0.0%	8,318
10	deep_breath	43.6% (218/500)	-0.4%	8,294
11	self_belief	42.0% (210/500)	-2.0%	8,274
12	verify_answer	41.0% (205/500)	-3.0%	8,160
13	emotional_stakes	38.4% (192/500)	-5.6%	7,825
14	rethink	38.2% (191/500)	-5.8%	7,755
15	decompose	37.6% (188/500)	-6.4%	7,837
16	negative_reminder	36.8% (184/500)	-7.2%	7,667

Figure 2: Impact vs Baseline (Diverging Bar)



Per-Subject Breakdown (Top 4 vs Baseline)

Figure 3: Per-Subject Accuracy Comparison



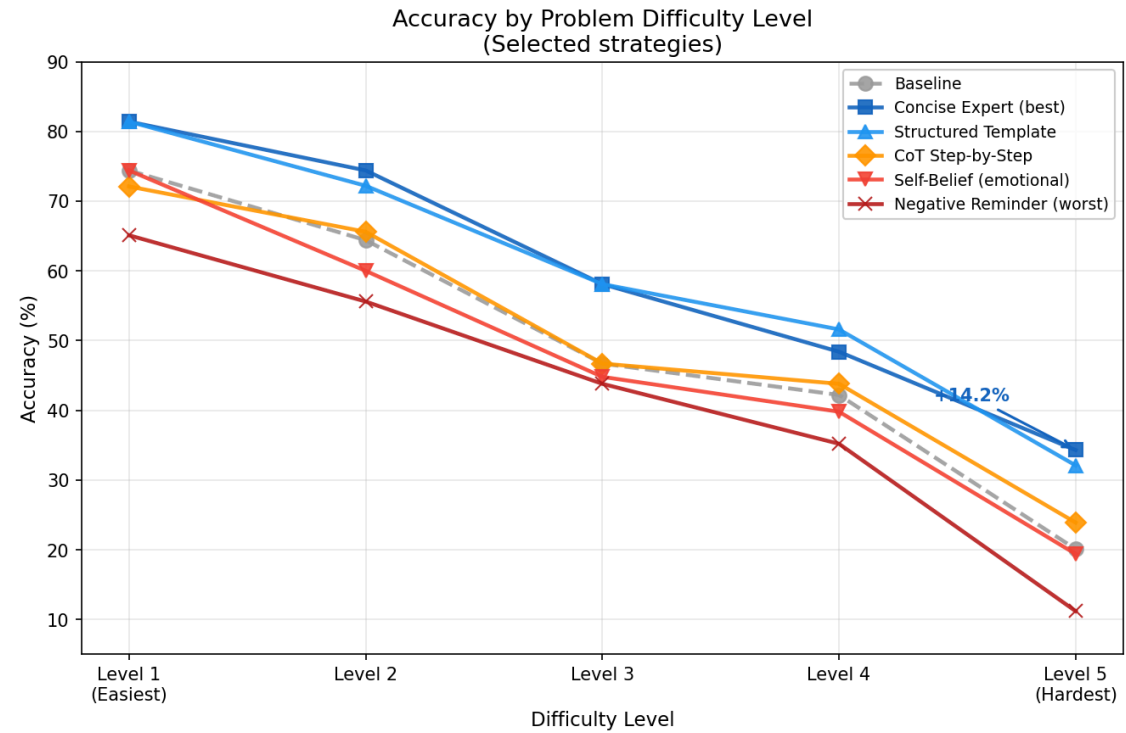
Subject	Baseline	concise_expert	structured_template	scratchpad	expe
---------	----------	----------------	---------------------	------------	------

Algebra	66.9%	73.4%	71.0%	73.4%	73.4%
Counting & Prob.	39.5%	50.0%	47.4%	44.7%	50.0%
Geometry	26.8%	36.6%	31.7%	29.3%	31.7%
Intermediate Alg.	21.6%	29.9%	35.1%	33.0%	25.8%
Number Theory	46.8%	71.0%	71.0%	71.0%	69.4%
Prealgebra	61.0%	68.3%	67.1%	65.9%	65.9%
Precalculus	19.6%	30.4%	32.1%	26.8%	28.6%

Notable gains: Number Theory shows the largest improvement (+24.2% for top strategies). Geometry and Precalculus also benefit substantially (+10-13%).

Per-Difficulty-Level Breakdown (concise_expert vs Baseline)

Figure 4: Accuracy by Problem Difficulty Level



Level	Baseline	Best (concise_expert)	Delta
1 (easiest)	74.4%	81.4%	+7.0%

2	64.4%	74.4%	+10.0%
3	46.7%	58.1%	+11.4%
4	42.2%	48.4%	+6.2%
5 (hardest)	20.1%	34.3%	+14.2%

Key insight: The hardest problems (Level 5) benefit the most from good prompting — a +14.2% absolute improvement.

Detailed Strategy Analysis

Tier 1: Winners (+8% to +10% over baseline)

1. concise_expert (54.2%) — "Focus on the key mathematical reasoning. Be concise but rigorous."

- Best overall accuracy
- Produces the **shortest** responses (7,538 chars) — less verbose, more focused reasoning
- Hypothesis: By asking for conciseness, the model avoids going down irrelevant tangents during its `<think>` phase

2. structured_template (54.0%) — System prompt defining 4-step structure (Understanding → Approach → Solution → Answer)

- Nearly tied with `concise_expert`
- Best on Intermediate Algebra (35.1%) and Precalculus (32.1%)
- Provides clear scaffolding that organizes the model's reasoning

3. scratchpad (53.0%) — "Solve using a scratchpad. Show all your work."

- Strong across all subjects
- Explicit "show your work" framing aligns well with the model's internal `<think>` process

4. expert_persona (52.2%) — System: "You are a brilliant mathematician..."

- Surprisingly effective despite literature claims that personas hurt
- Possible explanation: for a small 1.7B model, persona priming activates math-related knowledge more effectively than in larger models where it causes overconfidence

Tier 2: Modest Gains (+1% to +4%)

5. plan_and_solve_plus (48.2%) — "Extract variables, devise a plan, calculate intermediate variables"

- The explicit variable extraction step helps on structured problems
- +4.2% over baseline — the specific instructions add meaningful structure

6-7. cot_step_by_step (45.4%) & cot_think_carefully (45.2%)

- Classic "Let's think step by step" and "think more carefully" variants
- Minimal gain because Qwen3 already does CoT internally

Tier 3: No Improvement (±1%)

8-9. baseline (44.0%) & plan_and_solve (44.0%)

- Generic "devise a plan" without specific extraction steps adds nothing

10. deep_breath (43.6%) — "Take a deep breath and work step by step"

- The famous OPRO prompt — **slightly hurts** performance here
- Confirms 2024-2025 findings: emotional/relaxation framing doesn't help modern models

Tier 4: Harmful Prompts (-2% to -7%)

11. self_belief (42.0%) — "Think deep, believe in yourself!"

- -2.0% vs baseline
- Motivational fluff distracts from the task

12. verify_answer (41.0%) — "Solve, then verify your answer"

- -3.0% — asking for self-verification actually introduces second-guessing errors
- The model sometimes changes correct answers during "verification"

13. emotional_stakes (38.4%) — "Very important to my career. I believe in your ability!"

- -5.6% — emotional pressure degrades reasoning
- Confirms: emotional prompting hurts small models

14. rethink (38.2%) — "Solve, then re-examine and fix errors"

- -5.8% — similar to verify_answer but worse
- Self-correction in small models is unreliable

15. decompose (37.6%) — "Break into sub-problems, solve each"

- -6.4% — decomposition instructions confuse the model's natural reasoning flow
- May work better with few-shot examples showing how to decompose

16. negative_reminder (36.8%) — "Avoid common mistakes: sign errors, edge cases..."

- -7.2% — **worst performing strategy**
- Drawing attention to errors may prime the model to make them (ironic process theory)

Key Findings

Figure 5: Accuracy vs Response Verbosity

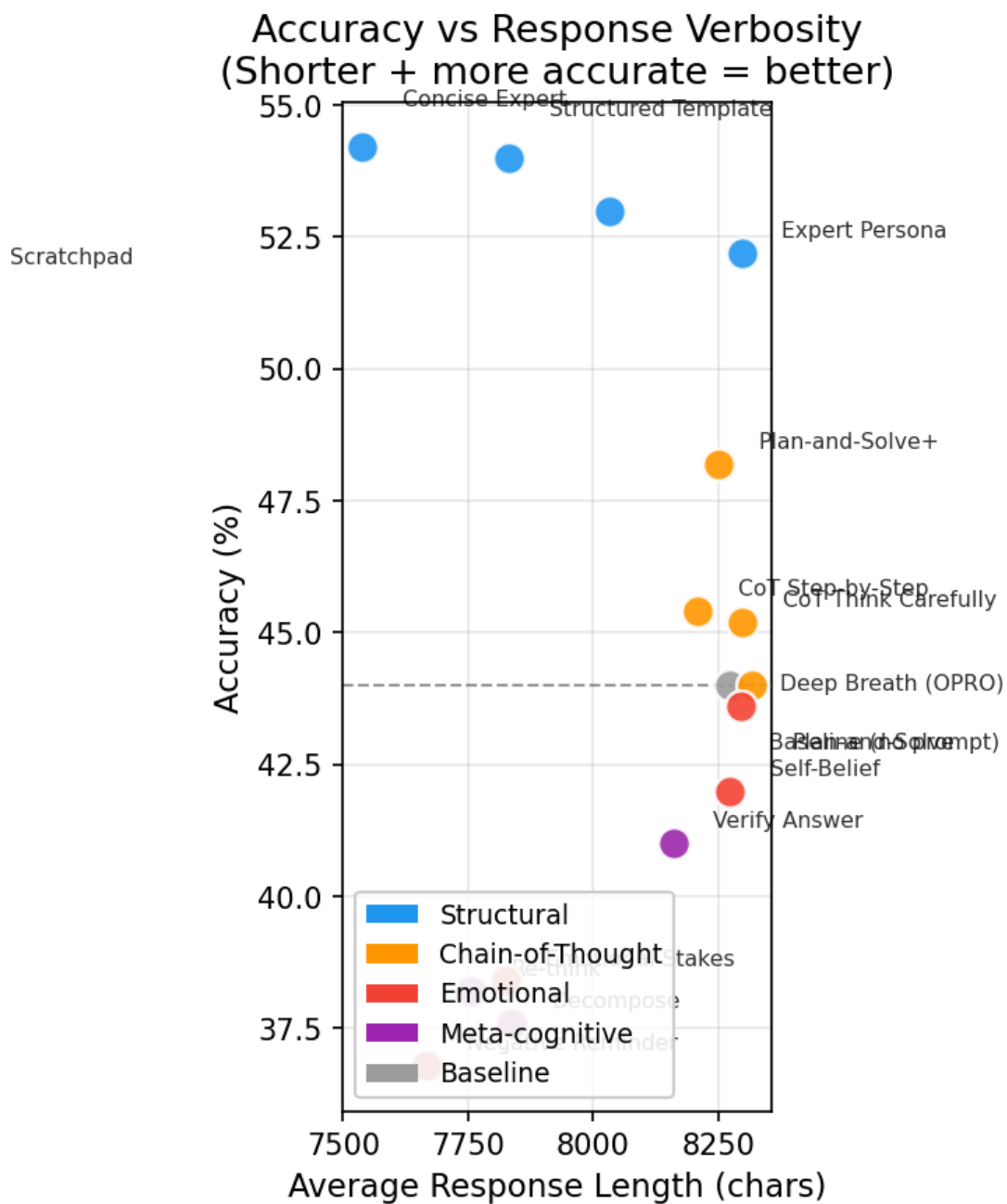
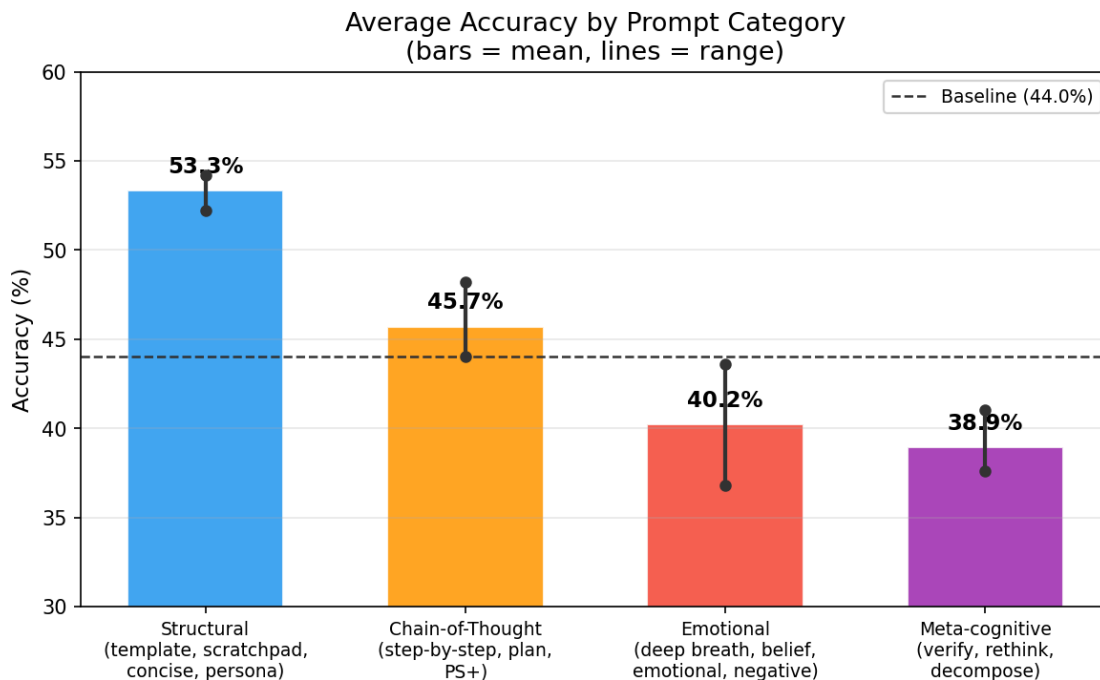


Figure 6: Average Accuracy by Prompt Category



1. Thinking models reduce the marginal value of explicit CoT

Qwen3-1.7B already generates chain-of-thought reasoning in `<think>` tags. Adding "Let's think step by step" provides only +1.4% because it's redundant. The model already thinks step by step.

2. Structural prompts beat motivational ones

The top 4 strategies all provide **structural guidance** (be concise, follow this template, use a scratchpad, adopt expert role). The bottom 4 all provide **motivational/emotional content** (believe in yourself, this is important, avoid mistakes). Structure helps; emotion hurts.

3. Conciseness is a feature, not a bug

The best strategy (concise_expert) produces the shortest responses. By asking for focused reasoning, the model avoids verbose tangents. **More tokens ≠ better reasoning** for small models.

4. Negative framing backfires

Telling the model what NOT to do (avoid mistakes, be careful of errors) consistently hurts performance. This aligns with psychological findings that negative instructions can prime the very behavior they warn against.

5. Self-verification is counterproductive for small models

Both "verify your answer" and "re-examine your solution" hurt accuracy by 3-6%. Small models lack the meta-cognitive ability to reliably self-correct — they're more likely to change a correct answer to an incorrect one.

6. Expert persona works for small models

Despite Google DeepMind's 2024 finding that expert personas hurt large models, they help Qwen3-1.7B (+8.2%). Smaller models may benefit from role priming that activates domain-specific knowledge, while larger models already have these capabilities activated.

7. Hardest problems benefit most from good prompting

Level 5 problems see +14.2% improvement from `concise_expert` vs baseline. Good prompting has the highest ROI on the hardest problems.

Recommended Prompts for Qwen3-1.7B Math Reasoning

Best Overall (54.2%):

```
{problem}
```

Focus on the key mathematical reasoning. Be concise but rigorous. Show only the essential steps needed to reach the answer.

Best with System Prompt (54.0%):

System:

You are a math problem solver. For each problem, follow this exact structure:

- Understanding**: State what is given and what is asked.
- Approach**: Describe your solution strategy.
- Solution**: Execute the solution step by step.
- Answer**: State the final answer in `\boxed{}`.

User: {problem}

Best Minimal (53.0%):

Please solve this problem using a scratchpad. Show all your work clearly.

```
{problem}
```

Scratchpad:

Experimental Setup Details

- Model**: Qwen3-1.7B (base, from `/workspace/models/Qwen3-1.7B`)
- Server**: SGLang with DP=2 on 2x NVIDIA H100 80GB
- Decoding**: Greedy (temperature=0.0), max_tokens=4096
- Concurrency**: 32 parallel requests
- Answer extraction**: `\boxed{}` pattern extraction with `math_verify` + normalized string comparison
- Time per strategy**: ~180-240 seconds (500 problems)

Artifacts

- Evaluation script: `experiments/prompting-tricks/eval_prompts.py`

- Per-strategy results: `experiments/prompting-tricks/results/*.jsonl`
 - Summary: `experiments/prompting-tricks/results/summary_final.json`
 - Figures: `logs/runs/20260316_063839-prompting-tricks-reasoning/reports/figures/`
 - Figure generation script: `experiments/prompting-tricks/generate_figures.py`
 - Deep research report: `logs/runs/20260316_063839-prompting-tricks-reasoning/reports/deep_research_report.md`
-

Conclusions

For **thinking models** (models with built-in CoT like Qwen3), traditional prompting tricks like "Let's think step by step" provide minimal benefit. Instead, **structural guidance** that shapes the reasoning format (structured templates, scratchpads) and **conciseness instructions** that prevent verbose tangents are the most effective strategies. Emotional/motivational prompting and self-verification instructions consistently degrade performance on mathematical reasoning tasks.

The practical takeaway: **tell the model HOW to structure its thinking, not HOW HARD to think.**